

Complexity Analysis

Before comparing algorithms we need a way to describe how their cost grows with input size, independent of the computer they run on.

Big-O Notation

Prerequisites: none beyond basic programming.

Big-O notation describes how the running time or memory of an algorithm grows as the input size n grows, ignoring constant factors and smaller terms. It usually describes the worst case.

Common classes, from fastest to slowest:

- $O(1)$, constant: looking up an array element by index or a key in a hash table.
- $O(\log n)$, logarithmic: binary search on a sorted array, which halves the search space each step.
- $O(n)$, linear: scanning a list once.
- $O(n \log n)$: efficient sorting algorithms such as merge sort.
- $O(n^2)$, quadratic: two nested loops over the same data, such as bubble sort.
- $O(2^n)$, exponential: trying every subset; naive recursive Fibonacci is close to this.

To find the complexity, count how the number of basic steps grows: a loop over n items inside another loop over n items gives n times n . Drop constants, so $3n + 5$ is $O(n)$.

Space complexity uses the same notation for memory. An algorithm can trade space for time, for example storing results in a hash table to avoid recomputing them. Big Omega and Big Theta describe lower and tight bounds.

Recursion

Prerequisites: Functions.

A recursive function calls itself on a smaller version of the same problem. Every recursive function needs a base case that returns without recursing, and a recursive case that moves toward the base case. Without a reachable base case, the function calls itself forever.

Factorial is the classic example: $\text{factorial}(n) = n \text{ times } \text{factorial}(n - 1)$, with $\text{factorial}(0) = 1$ as the base case.

Each call is placed on the call stack until it returns. Deep recursion can exhaust the stack; Python raises a `RecursionError` after about 1,000 nested calls by default.

Recursion fits naturally with problems that have recursive structure: trees, divide-and-conquer algorithms such as merge sort, and backtracking such as generating permutations.

Naive recursion can repeat work. Recursive Fibonacci recomputes the same values many times, giving exponential time. Storing results of previous calls, called memoization, reduces it to $O(n)$. This idea leads directly to dynamic programming. Any recursive algorithm can also be rewritten iteratively using an explicit stack.